

Hide menu

Introduction to Vector Fields

Painting a Vector Field

Video: Calculating a Vector from Position Data23 min

Reading: Textbook: Mouse Interaction10 min

Video: Vector from Mouse History7 min

Video: Fanning Vectors with Mouse History8 min

Practice Assignment: Practice Quiz: Vector Fields30 min

Neighboring Vectors and Particles

Textbook: Mouse Interaction

As we learn how to interact with this vector field using our mouse, the first step will be to translate the mouse's position into an index in the vector field. This is a common operation when working with grid-based systems or 2D arrays that are stored in a 1D list.

Understanding the Vector Field Structure

Assuming your vector field is a grid with a certain number of columns (cols) and rows (rows), each cell in this grid represents a vector and is stored in a flat list (1D list). The challenge is to find the index in this list that corresponds to the mouse's position.

INDEX FROM POSITION

The problem of converting a 2D coordinate into a 1D index in a list is a common task in various computational applications, especially when dealing with 2D grids or matrices.

```
#Example
x = int( x / (float(world_x) / cols) )
y = int( y / (float(world_y) / rows) )
index = (rows * x) + y
```

VECTOR FIELD

all_vectors[mouse_index]

INDEX FROM POSITION

The problem of converting a 2D coordinate into a 1D index in a list is a common task in various computational applications, especially when dealing with 2D grids or matrices.

```
#Example
x = int( x / (float(world_x) / cols) )
y = int( y / (float(world_y) / rows) )
index = (rows * x) + y
```

VECTOR FIELD

all_vectors[n]

Translate Mouse Position to Grid Coordinates

To interact with the vector field, you need to convert the mouse's x and y coordinates to the corresponding column and row in the grid.

We will add a function to our vector field class that will allow us to get a force based on the x and y positions of the mouse.

```
1 def get_force(self, x, y):
2     # Convert the x-coordinate to the corresponding column in the vector field
3     x = int(x / (float(self.world_x) / self.cols))
4     # Convert the y-coordinate to the corresponding row in the vector field
5     y = int(y / (float(self.world_y) / self.rows))
6
7     # Calculate the index in the flat list (1D array) based on the 2D grid coordinates
8     index = y * self.cols + x
9
10    # Debugging print statement: prints the calculated index and the grid coordinates
11    print("index: " + str(index) + " x: " + str(x) + ", y: " + str(y))
12
13    # Check if the index is within the range of the all_forces list
14    if 0 <= index < len(self.all_forces):
15        # Return the force vector at the calculated index
16        return self.all_forces[index]
17    else:
18        # If the index is out of range, raise an IndexError
19        raise IndexError("Vector index out of range.")
20
```

Track Mouse Movement

To paint the vector field with the mouse, we need to calculate the direction and magnitude of the mouse movement. This requires keeping track of the current and previous mouse positions.

In Processing, the current mouse position is given by **mouseX** and **mouseY**, and we can create variables to store the previous positions.

MOUSE HISTORY

To animate or simulate effects in interactive applications, it's often beneficial to know not just the current position of the mouse, but also the **direction** and **magnitude** of its movement.

This movement can be represented as a **vector**, computed using the current mouse **position** and its **previous position**.

MOUSE HISTORY

```
#Example
mouse_history.append( PVector(mouseX, mouseY) )

if(len(mouse_history) > 2):
    mouse_history.pop(0)

if(len(mouse_history) > 1):
    line(mouse_history[0].x, mouse_history[0].y,
         mouse_history[1].x, mouse_history[1].y)
```

```
1 # Function to calculate and maintain a history of mouse positions
2 def calc_mouse_history():
3     # Append the current mouse position as a PVector to the mouse_history list
4     mouse_history.append(PVector(mouseX, mouseY))
5
6     # If the length of mouse_history exceeds 2, remove the oldest entry
7     if len(mouse_history) > 2:
8         mouse_history.pop(0)
9
10    # If there are at least two points in mouse_history, draw a line between them
11    if len(mouse_history) > 1:
12        stroke(255) # Set the stroke color to white
13        # Draw a line between the first and second points in the mouse history
14        line(mouse_history[0].x, mouse_history[0].y,
15            mouse_history[1].x, mouse_history[1].y)
```

- This function manages a list called **mouse_history**, which stores the history of mouse positions as PVector objects.
- Each time the function is called, it adds the current mouse position to the **mouse_history** list.
- The function ensures that **mouse_history** only keeps the **two most recent mouse positions**. If there are more than two entries, it **removes the oldest one**.
- If there are at least two positions stored in mouse_history, it draws a line between these two points. This effectively creates a trail that follows the mouse movement.
- This could be used for drawing applications, visualizing mouse movement, or as an input for other functions or calculations that require knowledge of the mouse's path.

Modifying the Vector Field with the Mouse Data

Once we have the mouse data in vector form and we have the means to calculate the position of the mouse in the vector field, we can modify the vector field by completely overriding the value of a vector or by adding the vector of the mouse to the vector in the vector field. This last approach allows to create a 'paint' effect.

```
1 # Function to add a new force to a vector in the vector field
2 def add_force(self, index, new_force):
3     # Check if the index is within the bounds of the all_forces list
4     if index < len(self.all_forces):
5         # Add the new force to the existing force vector at the specified index
6         self.all_forces[index].v_force.add(new_force)
7
```

- Purpose:** The **add_force** function is intended to modify the force vector at a specified index in the **all_forces** list. The modification is done by adding a new force vector to the existing one.
- Parameters:** It takes two parameters: **index**, which specifies which vector in the field to modify, and **new_force**, which is the vector to be added.
- Safety Check:** The function first checks if the provided index is valid (i.e., within the bounds of the all_forces list).

In the following lessons, we will learn how to 'increase the brush size' by modifying many vectors simultaneously.

Go to next item

Completed